

Seminario 2

Realizado por: Rafael Fernando Rivas Cabrera, Diego Cabrera Martinez y Álvaro García Zafrá

Primera parte:

Asignatura	Categoría	Tecnologías	Lenguajes	Conceptos clave
FUNDAMENTOS DEL SOFTWARE	back-end	SHELL	BASH	Frameworks
PROGRAMACIÓN Y DIRECCIÓN ORIENTADA A OBJETOS	back-end, front-end (final)	NetBeans, RubyMine	java, ruby	MVC, frameworks, GUI
FUNDAMENTOS DE BASES DE DATOS	back-end	Oracle	SQL	SGBD
DISEÑO Y DESARROLLO DE SISTEMAS DE INFORMACIÓN	back-end y front-end	PYCHARM	python	Oracle
ADMINISTRACIÓN DE BASES DE DATOS	backend	Oracle	SQL	Oracle, SGBD
COMPUTACIÓN UBICUA E INTELIGENCIA AMBIENTAL	back-end front-end			middleware
DESARROLLO DE SISTEMAS DISTRIBUIDOS	back-end	Oracle	SQL	Frameworks, Middleware
INGENIERÍA DE SISTEMAS DE INFORMACIÓN	back-end front-end			Middleware
PROGRAMACIÓN WEB	back-end front-end		Java, PHP, HTML, CSS, JavaScript	XML, Ajax
SISTEMAS MULTIMEDIA	front-end		Java	Framework
SISTEMAS DE INFORMACIÓN BASADO EN WEB	front-end back-end	.NET	Java, PHP, HTML, CSS, JavaScript	Frameworks, XML, Ajax, W3C
SISTEMAS DE INFORMACIÓN PARA EMPRESAS	back-end	.NET	Java, python, php, xml	Business Logic

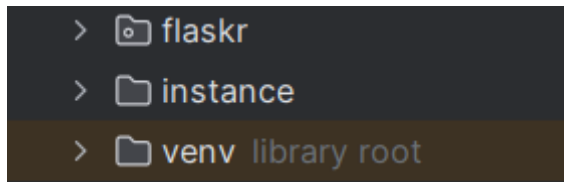
TECNOLOGÍAS WEB	back-end front-end		PHP, Java, HTML, CSS, JavaScript	XML, MVC
BASES DE DATOS DISTRIBUIDAS	back-end			SGBD
DESARROLLO BASADO EN AGENTES	front-end	JADE, MAGENTIX	HTML	Framework, middleware
DESARROLLO DE APLICACIONES PARA INTERNET	back-end front-end	jQuery, AJAX	SQL, HTML, CSS, JavaScript, python	MVC, frameworks, sistemas de información basados en web, aplicaciones web, DOM
INFRAESTRUCTURA VIRTUAL	back-end	git, github, HTTP	JavaScript, python, Perl	Servidores web
PROGRAMACIÓN TÉCNICA Y CIENTÍFICA	back-end front-end	Pandas, Scikit-learn, Numpy, Scipy	python	Desarrollo de aplicaciones WEB
PROGRAMACIÓN DE DISPOSITIVOS MÓVILES	back-end front-end	SDK		DOM
SISTEMAS COOPERATIVOS Y GESTIÓN DE CONTENIDOS	back-end		HTML, CSS, PHP, Javascript.	MVC

Segunda parte:

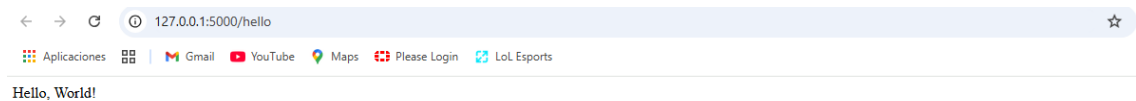
Para este apartado hemos seguido el tutorial de Flask, el cual nos enseña a hacer una página web en la que los usuarios pueden realizar posts. Asimismo, nos indica como hacer el sistema para registrarse e iniciar sesión posteriormente.

Cabe destacar que se utiliza una base de datos SQLITE, ya que éstas son más sencillas de acceder y gestionar en menor escala, se crea y almacena en local.

En primer lugar, hemos creado la estructura que se nos indica:



Procedemos a crear el primer archivo flaskr/___init___py e insertar el código especificado con el que haremos nuestro primer hola mundo.



A continuación, creamos el archivo encargado e la conexión con la base de datos en nuestro caso sqlite con las funciones encargadas de gestionar la base de datos:

```
1 usage
def get_db():
    if 'db' not in g:
        g.db = sqlite3.connect(
            current_app.config['DATABASE'],
            detect_types=sqlite3.PARSE_DECLTYPES
        )
        g.db.row_factory = sqlite3.Row

    return g.db

def close_db(e=None):
    db = g.pop(name='db', default=None)

    if db is not None:
        db.close()

1 usage
def init_db():
    db = get_db()

    with current_app.open_resource('schema.sql') as f:
        db.executescript(f.read().decode('utf8'))

@click.command('init-db')
def init_db_command():
    """Clear the existing data and create new tables."""
    init_db()
    click.echo('Initialized the database.')

sqlite3.register_converter(
    "timestamp", lambda v: datetime.fromisoformat(v.decode())
)
```

Asimismo, creamos el archivo sql encargado de crear las tablas:

```
DROP TABLE IF EXISTS user;
DROP TABLE IF EXISTS post;

CREATE TABLE user (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  username TEXT UNIQUE NOT NULL,
  password TEXT NOT NULL
);

CREATE TABLE post (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  author_id INTEGER NOT NULL,
  created TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  title TEXT NOT NULL,
  body TEXT NOT NULL,
  FOREIGN KEY (author_id) REFERENCES user (id)
);
```

Ahora necesitamos registrar las funciones encargadas de gestionar conexión con la base de datos en la instancia de la app por ello incluimos la siguiente función:

```
def init_app(app):
    app.teardown_appcontext(close_db)
    app.cli.add_command(init_db_command)
```

Y llamamos a la función create_app de __init__.py:

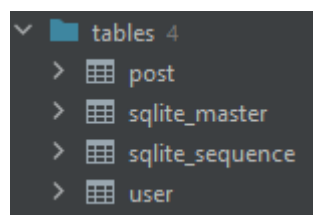
```
def create_app(test_config=None):
    # create and configure the app
    app = Flask(__name__, instance_name=instance_name)
    app.config.from_mapping(
        SECRET_KEY='dev',
        DATABASE=os.path.join(app.instance_path, 'flaskr.sqlite')
    )
    from . import db
    db.init_app(app)
```

Ahora creamos la base de datos desde la terminal con el comando que hemos añadido mediante app.cli.add.add_command(init_db_command) dentro de la función init_app(app) y se nos creará la base de datos con las tablas ya creadas.

```
(pythonProject10) PS C:\pythonProject10> flask --app flaskr init-db
Initialized the database.
```



A screenshot of a file explorer window showing a folder named 'instance'. Inside this folder, there is a file named 'flaskr.sqlite' which is highlighted with a blue selection bar.



A screenshot of a database table explorer window. It shows a folder named 'tables' with a count of 4. Inside this folder, there are four tables listed: 'post', 'sqlite_master', 'sqlite_sequence', and 'user'. Each table has a small icon next to it representing a table structure.

Las dos tablas sqlite se generan por defecto, así que no entraremos en detalle en ellas.

Ahora el tutorial nos explica primero que nada que son los blueprints, los cuales nos sirven para organizar y modularizar nuestras vistas y código. Creamos un nuevo archivo auth.py y definimos en el blueprint auth.

```
import functools

from flask import (
    Blueprint, flash, g, redirect, render_template, request, session, url_for
)
from werkzeug.security import check_password_hash, generate_password_hash

from flaskr.db import get_db

bp = Blueprint(name='auth', __name__, url_prefix='/auth')
```

Y lo registramos en la app:

```
def create_app(test_config=None):
    # create and configure the app
    app = Flask(__name__, instance_relative_config=True)
    app.config.from_mapping(
        SECRET_KEY='dev',
        DATABASE=os.path.join(app.instance_path, 'flaskr.sqlite'),
    )
    from . import db
    db.init_app(app)
    from . import auth
    app.register_blueprint(auth.bp)
```

Creamos la lógica del URL para registrarse:

```
@bp.route(rule='/register', methods=('GET', 'POST'))
def register():
    if request.method == 'POST':
        username = request.form['username']
        password = request.form['password']
        db = get_db()
        error = None

        if not username:
            error = 'Username is required.'
        elif not password:
            error = 'Password is required.'

        if error is None:
            try:
                db.execute(
                    "INSERT INTO user (username, password) VALUES (?, ?)",
                    (username, generate_password_hash(password)),
                )
                db.commit()
            except db.IntegrityError:
                error = f"User {username} is already registered."
            else:
                return redirect(url_for("auth.login"))

        flash(error)
```

El URL /register se asocia con la vista register. Registramos el input y validamos que los campos no estén vacíos, intentamos añadirlo a la base de datos y en caso de error lanzamos una excepción. En caso de que haya salido todo bien se insertará la fila en la base de datos y se redirigirá al usuario a la vista de log in.

Procedemos con la vista de login. Esta posee un esquema similar pero con la diferencia de que al introducir nombre y contraseña buscamos en la base de datos si existe el nombre, luego si este corresponde con la contraseña introducida y en caso de no haber ningún error cortamos la conexión y redirigimos a la ventana de index. En caso de haber algún error se le notifica. Cabe destacar que el id se guarda en mediante sesión en una cookie, lo que nos permitirá utilizar dicha información en otras vistas.

```
@bp.route(rule='/login', methods=('GET', 'POST'))
def login():
    if request.method == 'POST':
        username = request.form['username']
        password = request.form['password']
        db = get_db()
        error = None
        user = db.execute(
            "SELECT * FROM user WHERE username = ?", (username,)
        ).fetchone()

        if user is None:
            error = 'Incorrect username.'
        elif not check_password_hash(user['password'], password):
            error = 'Incorrect password.'

        if error is None:
            session.clear()
            session['user_id'] = user['id']
            return redirect(url_for('index'))

        flash(error)

    return render_template("auth/login.html")
```

Añadimos ahora la lógica de si existe una sesión utilizarla, lo cual es posible gracias a que guardamos el id en caso de haber iniciado sesión en una sesión:

```
@bp.before_app_request
def load_logged_in_user():
    user_id = session.get('user_id')

    if user_id is None:
        g.user = None
    else:
        g.user = get_db().execute(
            'SELECT * FROM user WHERE id = ?', (user_id,)
        ).fetchone()
```

Cabe destacar que `before_app_request` se ejecuta antes que la función de la vista, antes de que el usuario pueda ver nada.

Ahora vamos con `logout`, que limpia todas las sesiones y nos redirige a `index`

```
@bp.route('/logout')
def logout():
    session.clear()
    return redirect(url_for('index'))
```

Por si necesitas autenticación en otras vistas:

```
def login_required(view):
    @functools.wraps(view)
    def wrapped_view(**kwargs):
        if g.user is None:
            return redirect(url_for('auth.login'))

        return view(**kwargs)

    return wrapped_view
```

De momento, tenemos la lógica, pero nos falta todavía el apartado visual, por lo que el tutorial nos introduce las plantillas que nos permitirán tener un código en HTML más flexible y modularizado además de versátil.

Creamos un template base:

```
<!doctype html>
<title>{% block title %}{% endblock %} - Flask</title>
<link rel="stylesheet" href="{{ url_for('static', filename='style.css')}}">
<nav>
  <h1>Flask</h1>
  <ul>
    {% if g.user %}
      <li><span>{{ g.user['username'] }}</span>
      <li><a href="{{ url_for('auth.logout') }}">Log Out</a>
    {% else %}
      <li><a href="{{ url_for('auth.register') }}">Register</a>
      <li><a href="{{ url_for('auth.login') }}">Log In</a>
    {% endif %}
  </ul>
</nav>
<section class="content">
  <header>
    {% block header %}{% endblock %}
  </header>
  {% for message in get_flashed_messages() %}
    <div class="flash">{{ message }}</div>
  {% endfor %}
  {% block content %}{% endblock %}
</section>
```

Ahora para register:

```
{% extends 'base.html' %}

{% block header %}
  <h1>{% block title %}Register{% endblock %}</h1>
{% endblock %}

{% block content %}
  <form method="post">
    <label for="username">Username</label>
    <input name="username" id="username" required>
    <label for="password">Password</label>
    <input type="password" name="password" id="password" required>
    <input type="submit" value="Register">
  </form>
{% endblock %}
```

Para login:

```
{% extends 'base.html' %}

{% block header %}
  <h1>{% block title %}Log In{% endblock %}</h1>
{% endblock %}

{% block content %}
  <form method="post">
    <label for="username">Username</label>
    <input name="username" id="username" required>
    <label for="password">Password</label>
    <input type="password" name="password" id="password" required>
    <input type="submit" value="Log In">
  </form>
{% endblock %}
```


Con esto ya tendríamos un sistema funcional de registro e inicio de sesión:

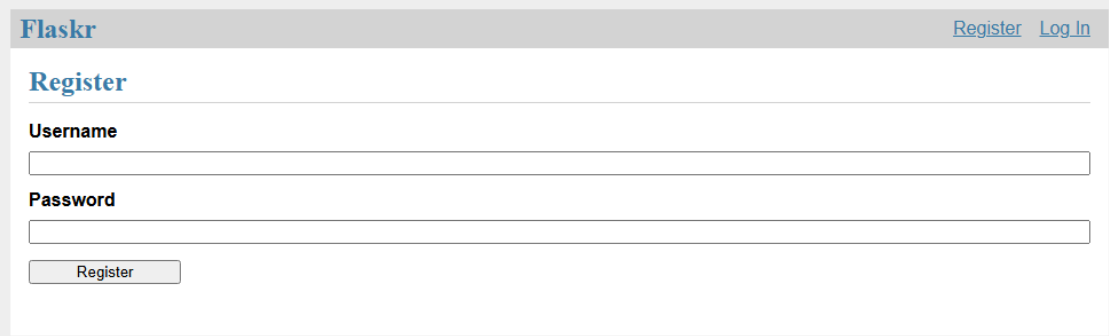
Flaskr

- [Register](#)
- [Log In](#)

Register

Username Password

No obstante, se ve bastante mal por lo que debemos mejorarlo usando el script CSS que nos indica el tutorial.



The image shows a web browser window with the title 'Flaskr'. In the top right corner, there are links for 'Register' and 'Log In'. The main content area is titled 'Register' and contains two input fields: 'Username' and 'Password'. Below the 'Password' field is a 'Register' button. The form is styled with a light blue header and a light gray border.

Y ahora ya se ve mucho mejor.

Procedemos ahora a seguir los casos para hacer el apartado de blog, lógica que al ser similar a la anteriormente expuesta no entraremos en detalle:

Creemos igual que anteriormente un blueprint y lo añadimos a la app:

```
from flask import (
    Blueprint, flash, g, redirect, render_template, request, url_for
)
from werkzeug.exceptions import abort

from flaskr.auth import login_required
from flaskr.db import get_db

bp = Blueprint(name: 'blog', __name__)
```

```
from . import blog
app.register_blueprint(blog.bp)
app.add_url_rule(rule: '/', endpoint='index')
```

Creamos la lógica de la función que obtendrá los posts de más reciente a más antiguo, Y su respectivo html:

```
@bp.route('/')
def index():
    db = get_db()
    posts = db.execute(
        'SELECT p.id, title, body, created, author_id, username'
        ' FROM post p JOIN user u ON p.author_id = u.id'
        ' ORDER BY created DESC'
    ).fetchall()
    return render_template('blog/index.html', posts=posts)
```

```
{% extends 'base.html' %}

{% block header %}
<h1>{% block title %}Posts{% endblock %}</h1>
{% if g.user %}
<a class="action" href="{{ url_for('blog.create') }}">New</a>
{% endif %}
{% endblock %}

{% block content %}
{% for post in posts %}
<article class="post">
<header>
<div>
<h1>{{ post['title'] }}</h1>
<div class="about">by {{ post['username'] }} on {{ post['created'].strftime('%Y-%m-%d') }}</div>
</div>
{% if g.user['id'] == post['author_id'] %}
<a class="action" href="{{ url_for('blog.update', id=post['id']) }}">Edit</a>
{% endif %}
</header>
<p class="body">{{ post['body'] }}</p>
</article>
{% if not loop.last %}
<hr>
{% endif %}
{% endfor %}
{% endblock %}
```

Continuamos la lógica de la vista para crear un post y su html:

```
def create():
    if request.method == 'POST':
        title = request.form['title']
        body = request.form['body']
        error = None

        if not title:
            error = 'Title is required.'

        if error is not None:
            flash(error)
        else:
            db = get_db()
            db.execute(
                'INSERT INTO post (title, body, author_id)'
                ' VALUES (?, ?, ?)',
                (title, body, g.user['id'])
            )
            db.commit()
            return redirect(url_for('blog.index'))

    return render_template('blog/create.html')
```

```
{% extends 'base.html' %}

{% block header %}
<h1>{% block title %}New Post{% endblock %}</h1>
{% endblock %}

{% block content %}
<form method="post">
  <label for="title">Title</label>
  <input name="title" id="title" value="{{ request.form['title'] }}" required>
  <label for="body">Body</label>
  <textarea name="body" id="body">{{ request.form['body'] }}</textarea>
  <input type="submit" value="Save">
</form>
{% endblock %}
```

Faltaría la opción para editar un post, para lo que tendremos que comprobar si el id de la sesión coincide con el del autor del post.

```
def get_post(id, check_author=True):
    post = get_db().execute(
        'SELECT p.id, title, body, created, author_id, username'
        ' FROM post p JOIN user u ON p.author_id = u.id'
        ' WHERE p.id = ?',
        (id,)
    ).fetchone()

    if post is None:
        abort(status=404, *args=f"Post id {id} doesn't exist.")

    if check_author and post['author_id'] != g.user['id']:
        abort(403)
```

```
@bp.route(rule='/<int:id>/update', methods=('GET', 'POST'))
@login_required
def update(id):
    post = get_post(id)

    if request.method == 'POST':
        title = request.form['title']
        body = request.form['body']
        error = None

        if not title:
            error = 'Title is required.'

    if error is not None:
        flash(error)
    else:
        db = get_db()
        db.execute(
            'UPDATE post SET title = ?, body = ?'
            ' WHERE id = ?',
            (title, body, id)
        )
        db.commit()
        return redirect(url_for('blog.index'))

    return render_template(template_name_or_list='blog/update.html', post=post)
```

```
{% extends 'base.html' %}

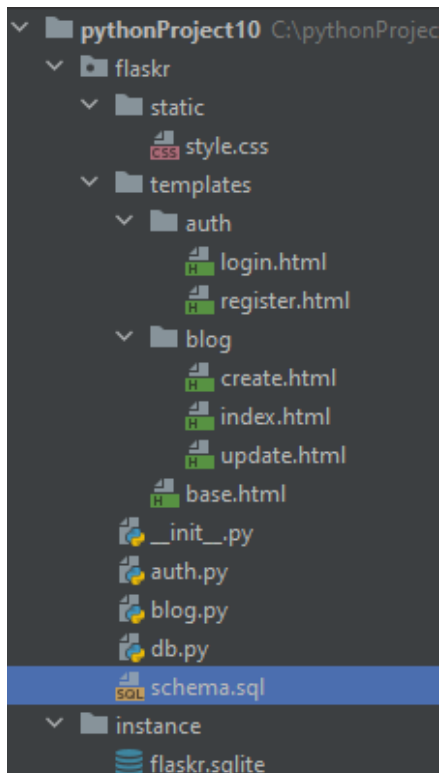
{% block header %}
<h1>{% block title %}Edit "{{ post['title'] }}" {% endblock %}</h1>
{% endblock %}

{% block content %}
<form method="post">
  <label for="title">Title</label>
  <input name="title" id="title"
    value="{{ request.form['title'] or post['title'] }}" required>
  <label for="body">Body</label>
  <textarea name="body" id="body">{{ request.form['body'] or post['body'] }}</textarea>
  <input type="submit" value="Save">
</form>
<hr>
<form action="{{ url_for('blog.delete', id=post['id']) }}" method="post">
  <input class="danger" type="submit" value="Delete" onclick="return confirm('Are you sure?');">
</form>
{% endblock %}
```

Por último, la función delete que carece de template ya que se accede desde la vista update y nos redirige a index

```
@bp.route(rule: '/<int:id>/delete', methods=('POST',))
@login_required
def delete(id):
    get_post(id)
    db = get_db()
    db.execute('DELETE FROM post WHERE id = ?', (id,))
    db.commit()
    return redirect(url_for('blog.index'))
```

Y ya habríamos terminado el tutorial quedando como se ve a continuación:



Página en estado inicial:

Flaskr

[Register](#) [Log In](#)

Posts

Registrarse:

Flaskr

[Register](#) [Log In](#)

Register

Username

alumno

Password

Register

Inicio de sesión:

Flaskr

[Register](#) [Log In](#)

Log In

Username

alumno

Password

Log In

Flaskr

alumno [Log Out](#)

Posts

[New](#)

Realizar Post:

Flaskr

alumno [Log Out](#)

New Post

Title

Mi primer post.

Body

Este es mi primer post.

Save

Editar post:

Edit "Mi primer post."

Title

Mi primer post.

Body

Este es mi primer post.

Save

Delete

Visualizar POSTS:

Flaskr	alumno Log Out
Posts	New
Mi primer post. by alumno on 2024-12-13 Este es mi primer post.	Edit